

2025—2026 学年第 2 学期

大作业报告

OpenHarmony 智能家居控制系统项目总结

课 程 名	软件体系结构与应用开发
课 程 号	B01121
项 目 名	OmniHome 智能家居控制系统
学生姓名	待填写
学生学号	待填写
专业班级	待填写
所在学院	计算学院
指导老师	罗荣良
实验报告日期	2026 年 6 月 26 日

说明：姓名、学号和班级信息可按个人提交情况补充替换。

1 大作业目的和要求

本项目围绕“基于 OpenHarmony 操作系统的家居设备控制系统”赛题展开，目标是在国产开源鸿蒙生态下实现一个可演示、可扩展、具备传输安全能力的智能家居控制原型。课程大作业要求不仅展示产品需求层如何解决用户痛点，也要求完整呈现软件开发层的设计、实现、测试和优化过程。

- 产品需求层：面向家庭主用户与普通家庭成员，解决智能家居入口分散、状态割裂、控制反馈不及时的问题。
- 软件开发层：围绕 OpenHarmony 客户端、Fastify 控制中心服务、共享设备契约和 SQLite 本地状态管理，构建完整的控制闭环。
- 文档表达层：结合系统截图、架构图、数据流图和测试表格，使评审者能够直观看到项目深度、完整度与工程化思路。

从赛题要求看，本项目重点满足门禁控制、灯光控制、温湿度监测、空调温度调节、HarmonyOS 适配以及传输安全设计等核心指标，同时保留后续接入更多家居设备和规则引擎的扩展空间。

2 项目来源、人员和具体分工

条目	内容
项目题目	OmniHome 智能家居控制系统
项目来源	软件杯 A9 赛题《基于 OpenHarmony 操作系统的家居设备控制系统》
项目获奖情况	当前版本以课程大作业提交为主，奖项情况待后续补充
赛题目标	在 OpenHarmony 环境下完成门禁、灯光、空调与环境监测等设备的集中控制，并兼顾网络协议传输安全
分工情况	当前文档按单人提交版本整理：需求分析、原型设计、前端开发、后端开发、测试文档与答辩材料由本人统筹完成；如为小组提交，可将本行替换为成员学号、姓名及具体职责

该赛题具有较强的现实价值。一方面，它呼应国产操作系统与信创生态建设需求；另一方面，它聚焦家庭物联网中最敏感的安全与控制问题，要求项目既要“能用”，也要“可信、可扩展、可说明”。

3 项目概要

OmniHome 将 OpenHarmony 客户端作为统一交互入口，把门禁、灯光、气候、摄像头、家庭动态、场景与自动化等能力集中到一个中控应用中。系统用户不需要在多个 App 之间频繁切换，即可查看全屋状态、远程控制设备并执行联动场景。

- 目标人群：家庭主用户、家庭成员，以及课程评审中关注系统架构与体验完整性的教师评委。
- 核心价值：统一控制入口、统一设备状态视图、统一安全命令链路、统一同步与推送机制。

- 产品风格：界面简洁、交互直接、反馈明确，重点突出全屋状态感知和关键设备的高频操作。

从需求层来看，系统解决了传统智能家居应用的三个主要问题：其一是控制入口分散，用户难以快速完成跨设备操作；其二是设备状态分散，用户缺少家庭整体运行状态的统一视角；其三是控制结果不可追踪，很多简单原型只会调用接口，但缺少历史留痕、异常反馈与同步闭环。

因此，本项目并非停留在“页面演示”，而是尝试构建一个具备产品思维和工程思维的智能家居中控原型：既能展示场景价值，也能解释技术实现与系统边界。

4 项目详细方案

4.1 总体架构方案

系统采用“OpenHarmony 客户端应用层 + Fastify 控制中心服务层 + 共享契约层 + 数据与模拟设备层”的分层结构。前端负责界面展示、用户交互和本地缓存；后端负责设备注册、命令执行、场景编排、同步服务和 WebSocket 广播；共享契约层用于统一前后端模型；SQLite 与模拟设备层则负责状态持久化和原型闭环。

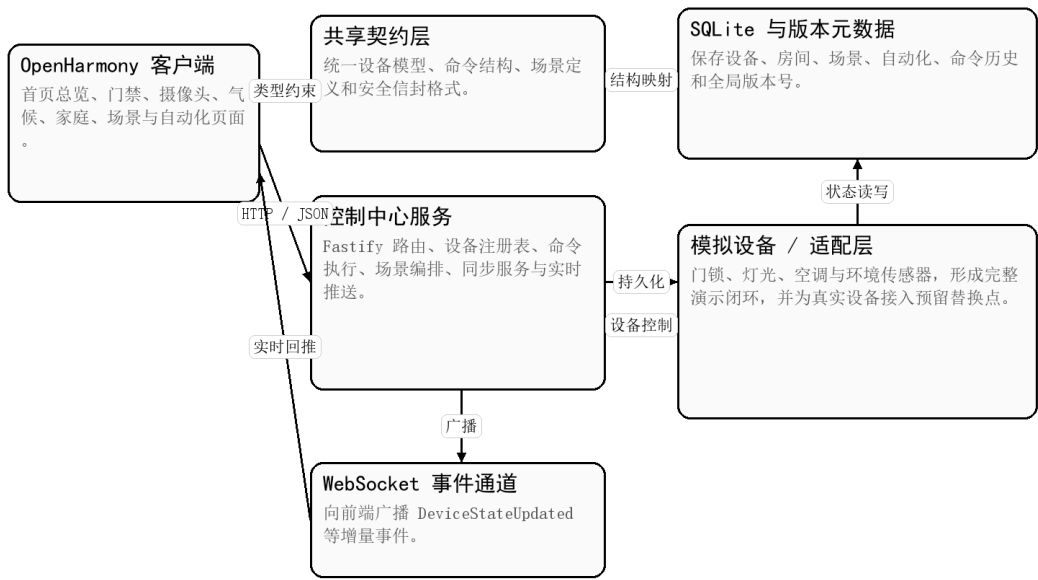


图 4-1 系统总体架构图

4.2 需求到功能的映射

赛题需求	系统设计方案	达成效果
房门开关控制	门禁模块 + 数字钥匙页 + 安全签名命令链路	可进行远程开锁、临时访客授权与状态回显
房间灯光控制	首页灯光卡片 + 房间灯光面板 + 设备指令接口	支持灯光开关、亮度与色温控制

温湿度监测	环境摘要卡片 + 气候控制页 + 后端环境数据接口	实时展示温湿度与空气质量，并可触发数据演示
空调设备控制	空调控制面板 + 适配器抽象层 + 模式切换接口	支持开关机、目标温度调节与运行模式切换
兼顾协议传输安全	HMAC 命令签名 + nonce 重放保护 + TLS 预留	保证原型具备可解释的安全控制链路
良好扩展性	共享契约包 + 路由模块化 + 版本同步机制	便于后续接入更多设备与自动化规则

4.3 关键业务流程方案

- 1. 用户进入首页后，客户端通过摘要接口和设备接口加载全屋状态，并读取本地缓存提高首屏响应速度。
- 2. 当用户操作灯光、门锁或空调时，前端先调用签名接口获取安全信封，再向命令接口提交正式控制请求。
- 3. 控制中心在完成验签和重放保护后，调用模拟设备或适配器执行命令，更新数据库状态和命令历史。
- 4. 服务端将状态变化通过 WebSocket 主动广播给前端，必要时前端也可通过 /api/sync 完成增量补偿同步。
- 5. 用户可在通知页和家庭动态页查看执行结果、家庭广播和异常反馈，从而形成可追踪的使用闭环。

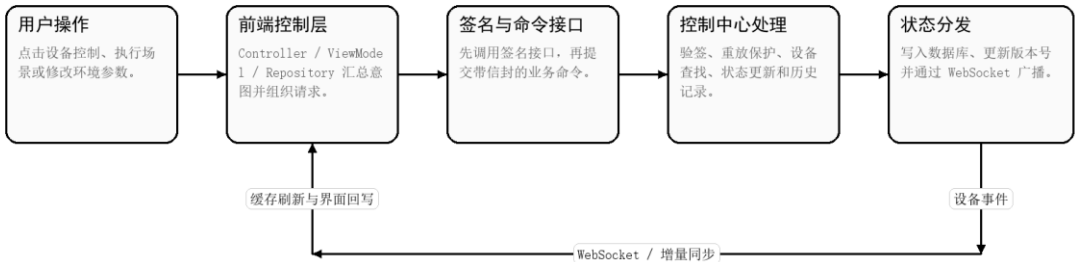


图 4-2 核心控制与同步数据流图

4.4 交互体验方案

为提高答辩展示效果和产品可用性，前端界面采用模块化导航结构：首页承担全局状态入口，门禁、摄像头、气候、家庭、自动化和通知分别对应垂直业务模块。高频操作尽量在一到

两次点击内完成，关键状态以卡片形式突出显示，异常情况则通过明显的状态文案和历史记录反馈给用户。

- 首页突出在线设备数、灯光数量、当前温湿度和空气质量，便于评审快速理解系统业务范围。
- 门禁、空调等高风险或高频模块采用单独页面，减少误操作并增强演示针对性。
- 自动化与场景页体现系统不仅能“单设备控制”，还能支持多设备联动与规则化控制。

5 系统实现

5.1 前端实现

前端位于 `apps/openharmony-control`，采用 ArkTS 编写，围绕 `pages`、`views`、`components`、`viewmodel`、`services` 和 `model` 六类模块组织代码。页面不直接拼接业务请求，而是通过 `ViewModel` 和 `Repository` 封装控制意图与数据访问逻辑，从而降低 UI 与网络层的耦合程度。

OmniHome 主要界面组合图

将首页、门禁、环境与自动化界面集中展示，便于在课程报告中进行整体说明。

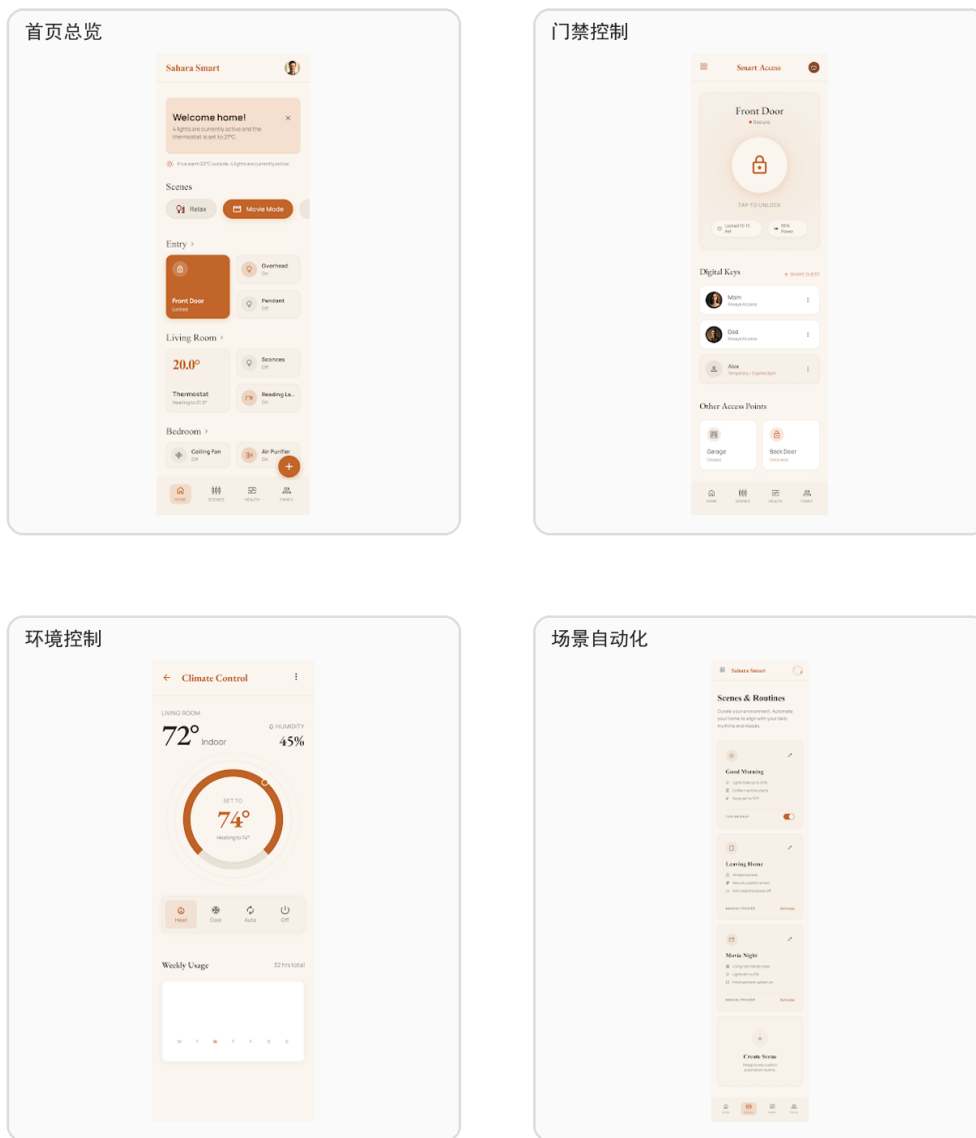


图 5-1 OmniHome 主要界面组合图

5.2 后端实现

后端位于 `services/control-center`，基于 `Fastify` 构建控制中心服务。系统通过 `devices`、`access`、`camera`、`climate`、`family`、`commands`、`scenes`、`automations`、`rooms`、`sync` 和 `websocket` 等模块化路由提供接口能力，并维护设备注册表、场景注册表、命令历史和数据库服务，形成完整的业务中枢。

- 设备契约统一：packages/device-contract 中定义 DeviceDescriptor、DeviceState、SceneDescriptor、SignedCommandEnvelope 等共享模型。
- 本地持久化：better-sqlite3 驱动 SQLite，维护 devices、rooms、scenes、automations、history 和 metadata 等核心数据表。
- 一致性保障：通过 global_version 与各表 version 字段支撑 /api/sync 增量同步接口。
- 实时反馈：通过 WebSocket 将 DeviceStateUpdated 等事件广播给客户端，减少纯轮询造成的延迟。

5.3 安全与可扩展实现

赛题明确强调协议传输安全性，因此本项目在原型阶段就加入了安全命令链路设计。客户端不会直接裸发控制命令，而是先向 /api/demo/sign-command 请求 HMAC-SHA256 签名，再向 /api/commands 提交带 nonce 与时间戳的安全信封。服务端通过 ReplayGuard 防止命令重放，并预留 TLS_CERT_PATH 与 TLS_KEY_PATH，支持后续迁移至 HTTPS 部署环境。

在扩展性方面，项目将设备控制能力和具体硬件实现解耦，当前通过模拟设备完成闭环验证，后续可在不重写上层页面和控制逻辑的前提下，将模拟器替换为真实设备网关或第三方物联网平台适配层。

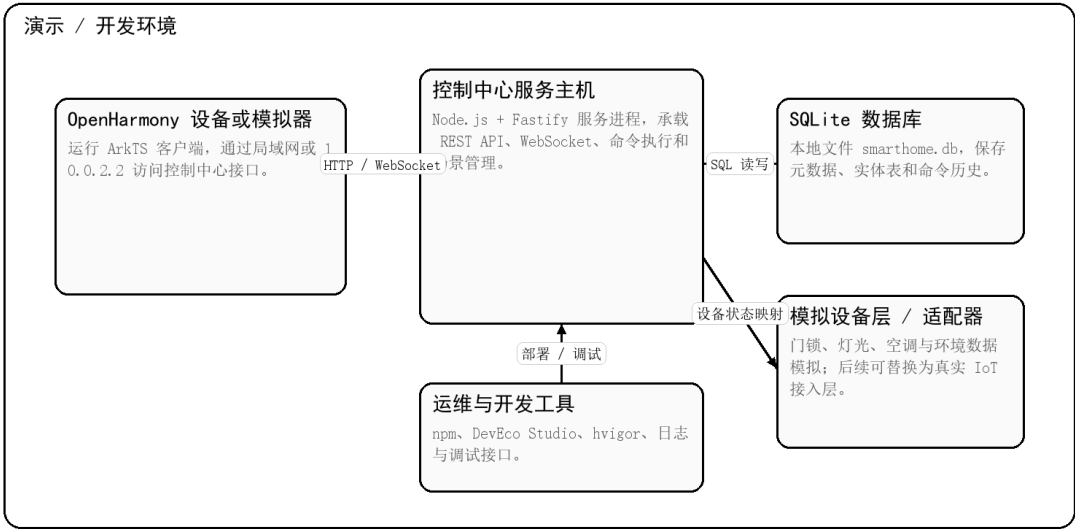


图 5-2 系统部署结构图

6 系统测试

测试部分围绕“功能完整度、控制正确性、同步正确性、安全性与工程可验证性”展开。项目既验证了设备控制主链路，也验证了异常拒绝、离线演示和契约一致性，确保文档展示内容与仓库实现保持一致。

测试领域	测试内容	预期结果与证据
共享契约测试	校验 access、camera、climate 等共享契约结构	device-contract 相关测试通过，保证前后端数据模型一致
设备命令测试	门锁解锁、灯光开关、环境状态读取	command-routes.test.ts、environment-routes.test.ts 验证控制与状态更新正确
业务接口测试	门禁、摄像头、家庭、气候、场景与自动化接口	对应 routes 测试通过，接口响应结构稳定
安全测试	未签名命令、重放命令、强制未授权模式	服务端返回 COMMAND_UNAUTHORIZED，体现安全拒绝链路
同步与实时测试	版本同步接口与 WebSocket 事件	sync.test.ts、websocket.test.ts 验证增量同步与实时回写
整体自动化结果	npm test 与 npm run typecheck	截至 2026-06-04，共 49 项测试通过，类型检查通过

根据现有测试报告，@smart-home/device-contract 的 9 项契约测试与 @smart-home/control-center 的 40 项服务测试均已通过，共计 49 项自动化测试通过。与此同时，DevEco/HAP 打包在当前环境下仍受 SDK 完整性与路径限制影响，因此报告中如实标注了该项验证边界，没有夸大系统已完成的能力。

- 测试覆盖了功能正确性，也覆盖了异常路径与安全路径，保证答辩时能够展示“系统如何失败、如何反馈”。
- 通过命令历史与通知页回显，可以把测试结果以用户可见的方式呈现出来，增强说服力。
- 本地缓存、增量同步和 WebSocket 共同验证了系统并非单纯页面原型，而是具有状态一致性设计的中控系统。

7 项目创新点

- 国产系统生态适配：以 OpenHarmony 为应用承载平台，呼应赛题对国产开源鸿蒙生态建设的要求。
- 控制闭环完整：从前端操作、命令签名、后端验签、设备执行、数据库写回到 WebSocket 广播，链路完整且可解释。
- 共享契约设计：通过独立的 device-contract 包统一前后端语义，减少联调成本并增强可扩展性。
- 同步机制工程化：不仅有 REST 查询接口，还引入本地 SQLite 缓存、版本增量同步和实时事件推送。
- 异常演示能力：支持设备离线、安全拒绝和环境变化演示，便于课程答辩中展示系统对真实问题的处理思路。
- 设备适配抽象：当前使用模拟设备完成演示闭环，同时为后续真实门锁、空调和传感器接入保留替换接口。

这些创新点并不追求“堆技术名词”，而是围绕课程评分点和真实可演示性展开：既体现软件体系结构课程对分层、解耦、扩展性的要求，也体现产品型项目对用户体验与场景价值的关注。

8 总结和心得

通过本次大作业，我对“软件体系结构”不再停留在概念层面，而是真正把分层设计、共享契约、缓存同步、命令安全和模块化扩展落实到了一个可运行的项目中。项目实践表明，一个好的智能家居系统不只是能把页面做出来，更重要的是让状态、控制、反馈和异常处理形成稳定闭环。

在需求层，我更加理解了为什么用户真正需要的是“统一入口和清晰反馈”，而不是更多按钮；在开发层，我体会到前后端边界、数据一致性和安全设计如果前期没有想清楚，后期会很难补救。这次项目让我进一步认识到，体系结构设计价值不在于画图本身，而在于它是否能指导团队持续迭代、降低耦合、支撑扩展。

如果后续继续完善本项目，我希望优先推进三个方向：一是接入真实 IoT 设备或网关，提升系统真实性；二是补充用户身份认证与权限体系，增强安全完整性；三是增强自动化规则引擎，让系统从“可控”进一步走向“可智能协同”。整体来看，本项目已经达到了课程大作业对产品需求分析、软件设计实现、测试验证和创新表达的综合要求。